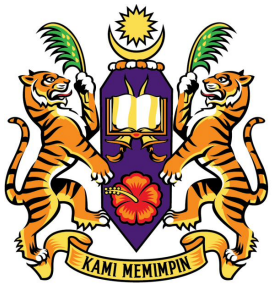


UNIVERSITI SAINS MALAYSIA

Universiti Sains Malaysia

School of Electrical and Electronic Engineering

EEM 441

Control, Instrumentation & FPGA Laboratory

Laboratory Manual

Experiments 1--10

Contents

EEM 441 --- Control, Instrumentation & FPGA Laboratory

Experiment 1

DC Motor / PID Control --- Speed

Objectives

1. To study the open-loop speed control of a DC motor.
2. To study closed-loop speed control using P, PD, PI, and PID controllers.

Equipment / Software Required

- Controller kit
- Cathode ray oscilloscope
- BNC connectors with leads
- Multimeter

Procedure

(a) Open-loop control of DC motor

1. Switch on the supply and measure all constant voltages; calibrate variable voltages in terms of voltage and angle (degrees).
2. Disconnect all cable wires from the hardware module.
3. Wire the circuit shown in Figure **??** for open-loop control.

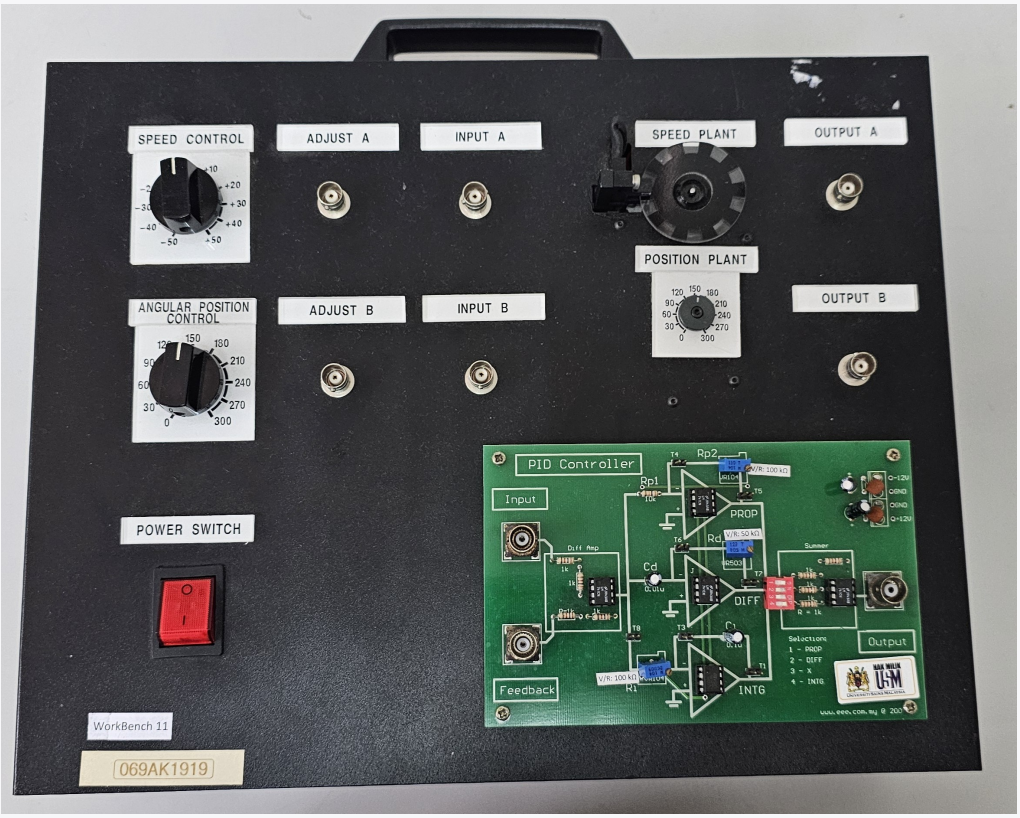


Figure 1: DC Motor / PID Control trainer, used for Experiments 1 and 2. The panel provides Speed Control and Angular Position Control knobs, Adjust A/B and Input A/B BNC connections, the Speed Plant and Position Plant motor assembly, Output A/B, and the PID Controller daughterboard (PROP, DIFF, INTG, and Summer stages).

4. Connect *Output A* of the motor plant to Channel 2 of the oscilloscope.
5. Connect *Adjust A* of the motor plant to *Input A* (speed control of the plant) and to Channel 1 of the oscilloscope (only positive voltages are considered).
6. Turn ON the power switch.
7. Slowly turn potentiometer-A clockwise (0 to $+50$) and anti-clockwise (0 to -50). Observe and describe the motor operation.
8. Disconnect *Adjust A* and connect the motor *Input A* to *Adjust B* (only positive voltages are considered).
9. Vary supply B from $30°$ to $270°$ in steps of $30°$ and study the variation in reference speed (voltage) and output speed (voltage). Plot the error curve.
10. Comment on your observations and results.

Position	V_{in} (Channel I)	V_{out} (Channel II)	Error ($V_{out} - V_{in}$)	Remarks
$+270$				
$+240$				
$+210$				
$+180$				
$+150$				
$+120$				
$+90$				
$+60$				
$+30$				

Table 1: Open-loop speed response.

(b) Closed-loop control of speed of DC motor

(i) Using P-controller with feedback through optical encoder

1. Select the P-controller by setting PROP on the controller (DIP switch 1) to ON.
2. Connect *Adjust B* of the plant to the input of the PID controller.
3. Connect *Output A* of the plant to the controller feedback.
4. Connect the controller output to *Input A* of the plant.
5. Turn potentiometer-B of the motor plant to a middle position (e.g. $+180$).
6. Adjust trimmer R_{p2} (varies K_p) to obtain an optimum response (minimum steady-state error).
7. Turn potentiometer-B to vary the speed of the motor. Record the input and output voltages for various positions in tabular form.

(ii) Speed control with PI-controller

1. On the PID controller card, set trimmers R_p and R_d to minimum (counter-clockwise) while R_i is set to maximum (clockwise).
2. Select the PI-controller by setting PROP and INTG to ON.
3. Slowly vary trimmer R_i to obtain an optimum output response (minimum steady-state error). If gain is insufficient (or overshoots), adjust R_{p2}.
4. Turn potentiometer-B to vary the speed of the motor. Record the input and output voltages for various positions (Table **??**). Repeat step 3 if necessary to fine-tune the response.
5. Compare the experimental results for the P-controller and the PI-controller.

(iii) Speed control with PD-controller

1. Switch on the supply and measure all constant voltages; calibrate variable voltages in terms of voltage and angle.
2. On the PID controller card, set trimmers R_p and R_d to minimum (counter-clockwise) while R_i is set to maximum (clockwise).
3. Select the PD-controller by setting PROP and DIFF to ON; all other switches OFF.
4. Vary supply B from $30°$ to $270°$ in steps of $30°$ and study the variation in reference speed (V_{in}) and output speed (V_o). Plot the error curve.
5. Study the effect of varying R_d on the output.

(iv) Speed control with PID-controller

1. Adjust potentiometer-B to 2 V (around $+180°$).
2. Set trimmer R_{p2} to minimum to reduce the damping ratio.
3. Set trimmer R_i to minimum. The output should start to vibrate.
4. Slightly increase R_i until the system starts oscillating between two points (pure oscillation).
5. Slightly increase R_{p2} to bring the system to a damped condition; it should oscillate for a few cycles.
6. Apply a small force to hold the motor (the oscilloscope will show the motor output drop to 0 V); observe and describe the response when the force is released.
7. Select the PID-controller by setting PROP, INTG, and DIFF to ON.
8. Slowly vary R_d to obtain the optimum response.
9. Compare its operation with the P, PI, and PD controllers under disturbance.

10. As before, take observations by varying the applied voltage.

Position	V_{in} (Channel I)	V_{out} (Channel II)	Error ($V_{out} - V_{in}$)	Remarks
+270				
+240				
+210				
+180				
+150				
+120				
+90				
+60				
+30				

Table 2: Closed-loop speed response.

Report

1. Compare the performance of P, PI, PD, and PID controllers.
2. What will happen if all types of controllers are connected in series?
3. What are the different types of encoders? Explain the working principle of an optical encoder with the help of suitable diagrams.

EEM 441 --- Control, Instrumentation & FPGA Laboratory

Experiment 2

DC Motor / PID Control --- Position

Objectives

1. To study open-loop position control of a DC motor.
2. To study feedback (closed-loop) control using P, PI, PD, and PID controllers.
3. To implement position control of a DC motor using a potentiometer and a resistance-to-voltage converter.

Equipment / Software Required

- Controller kit
- Cathode ray oscilloscope
- BNC connectors with leads
- Multimeter

Procedure

(a) Position control without feedback

1. Connect *Output B* of the motor plant to Channel 2 of the oscilloscope.
2. Connect *Adjust A* of the motor plant to *Input B* (position control) of the plant and to Channel 1 of the oscilloscope.
3. Turn on the power switch.
4. Slowly turn potentiometer-A clockwise (0 to $+50$) and anti-clockwise (0 to -50). Observe and describe the motor operation. Can you control the angular position of the motor?
5. Turn off the motor by disconnecting *Input B*.

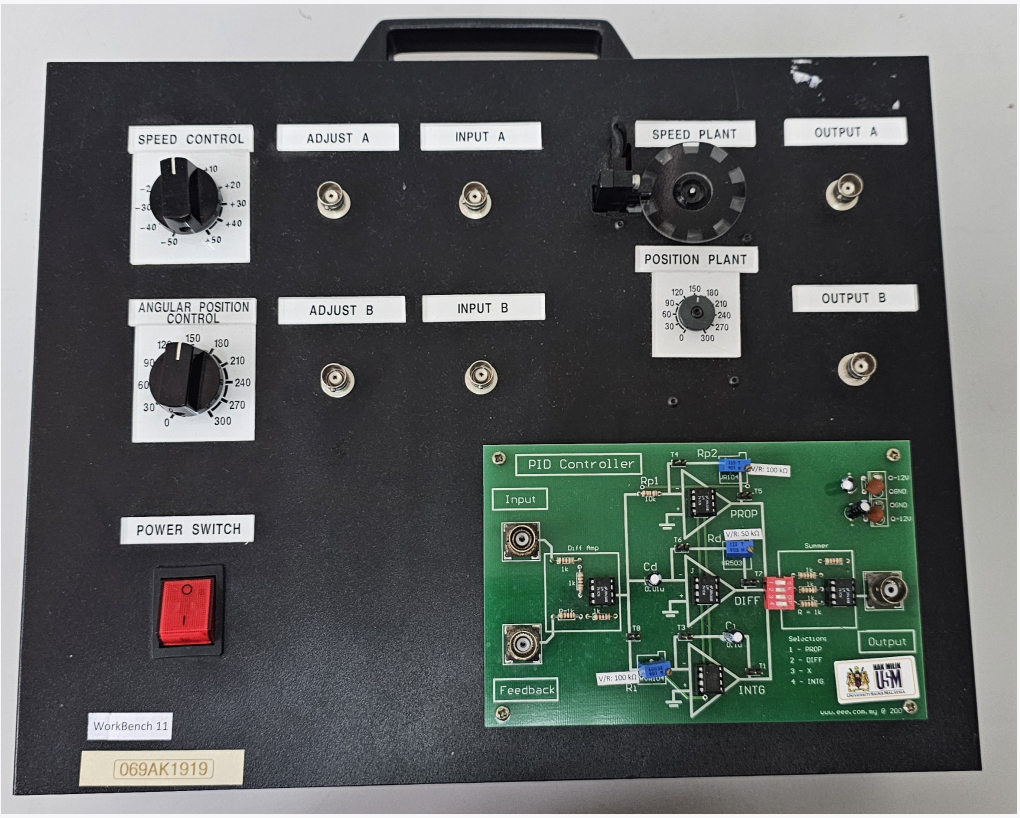


Figure 2: DC Motor / PID Control trainer, used for Experiments 1 and 2. The panel provides Speed Control and Angular Position Control knobs, Adjust A/B and Input A/B BNC connections, the Speed Plant and Position Plant motor assembly, Output A/B, and the PID Controller daughterboard (PROP, DIFF, INTG, and Summer stages).

(b) Closed-loop position control

(i) Position control with proportional feedback control

The motion of a motor can be converted into rotational position with the help of a suitable gearbox. The electrical signal, proportional to (angular) position, is obtained with the help of a circular potentiometer of good accuracy. This signal is fed back to the summer to extract an error signal, which is fed to the controller to obtain optimum performance. Different controllers may be tried for this case, to study them: use the PID Controller daughterboard shown in Figure **??**, selecting the desired stage(s) via its DIP switch.

1. Select the P-controller by setting PROP on the controller (DIP switch 1) to ON; all other switches OFF.
2. Connect *Adjust B* of the plant to the *input* of the PID controller.
3. Connect *Output B* of the plant to *Feedback* of the controller.
4. Connect the controller *Output* to *Input B* of the plant.
5. Turn potentiometer-B of the motor plant to a middle position (e.g. $+180$).
6. Adjust trimmer R_{p2} (varying K_p) to obtain an optimum response (minimum steady-state error).
7. Turn potentiometer-B to vary the position of the motor. Can the position be controlled in this case?
8. Record the input and output voltages for various positions in Table **??**.

Position	V_{in} (Channel I)	V_{out} (Channel II)	Error ($V_{out} - V_{in}$)	Remarks
$+270$				
$+240$				
$+210$				
$+180$				
$+150$				
$+120$				
$+90$				
$+60$				
$+30$				

Table 3: Position control response.

(ii) Position control with feedback and PI-controller

1. Select the PI-controller by setting PROP and INTG to ON.
2. Connect *Adjust B* of the plant to the *input* of the PID controller.
3. Connect *Output B* of the plant to *Feedback* of the controller.
4. Connect the controller *Output* to *Input B* of the plant.
5. Turn potentiometer-B of the motor plant to a middle position (e.g. $+180$).
6. Slowly vary the respective trimmers R_{p2} and R_i to obtain an optimum output response.
7. Turn potentiometer-B to vary the position of the motor. Record the input and output voltages for various positions.

(iii) Position control with PD-controller

1. Select the PD-controller by setting PROP and DIFF to ON; all other switches OFF.
2. Connect *Adjust B* of the plant to the *input* of the PID controller.
3. Connect *Output B* of the plant to *Feedback* of the controller.
4. Connect the controller *Output* to *Input B* of the plant.
5. Turn potentiometer-B of the motor plant to a middle position (e.g. $+180$).
6. Slowly vary the respective trimmers R_{p2} and R_d to obtain an optimum output response.
7. Turn potentiometer-B to vary the position of the motor. Record the input and output voltages for various positions.

(iv) Closed-loop control of position of motor using PID controller

1. Select the PID-controller by setting PROP, INTG, and DIFF to ON.
2. Connect *Adjust B* of the plant to the *input* of the PID controller.
3. Connect *Output B* of the plant to *Feedback* of the controller.
4. Connect the controller *Output* to *Input B* of the plant.
5. Turn potentiometer-B of the motor plant to a middle position (e.g. $+180$).
6. Adjust the value of trimmers R_{p2}, R_i, and R_d to obtain an optimum response (minimum steady-state error, quick operation, without overshoot or oscillations).
7. Turn potentiometer-B to vary the position of the motor.
8. Record the input and output voltages for various positions in Table **??**.

Position	V_{in} (Channel I)	V_{out} (Channel II)	Error ($V_{out} - V_{in}$)	Remarks
$+270$				
$+240$				
$+210$				
$+180$				
$+150$				
$+120$				
$+90$				
$+60$				
$+30$				

Table 4: Closed-loop PID position response.

Report

1. Briefly explain the working of circuits used for the conversion of resistance to voltage.
2. Describe in detail the applications of position control.

EEM 441 --- Control, Instrumentation & FPGA Laboratory

Experiment 3

Introduction to Altera Quartus II Software

Objectives

1. To use schematic entry as a design method in Altera Quartus II.
2. To use Verilog as a design entry method in Altera Quartus II.
3. To combine schematic and Verilog as a single design entry in Altera Quartus II.
4. To simulate the circuit using Altera Quartus II.
5. To download the design onto an Altera DE2 board.

Equipment / Software Required

- Altera Quartus II software
- Altera DE2 board

Procedure

Activity 1 — Schematic entry

Objective: use schematic entry to design a 4-to-1 multiplexer in Quartus II.

1. Open Quartus II and start the New Project Wizard.
   - Working directory: `fpga`; project name: `activity1`; top-level entity: `mux_sch`.
   - Device family: Cyclone II, device `EP2C35F672C6N`.
   - Accept the default EDA tool settings (synthesis, simulation, timing analysis) and finish.
2. Create a new Block Diagram/Schematic file.
3. Place three 3-input AND gates and one 4-input OR gate from the primitives library, and wire them as shown in Figure **??** using rubberbanding to connect components.
4. Add and name the input pins (`i0–i3`, `s0`, `s1`) and the output pin (`out`).
5. Save as `mux_sch.bdf` and start compilation.
6. Create a Vector Waveform File to simulate the design: insert input nodes `i0–i3`, `s0`, `s1`, and output node `out`.
7. Assign logic values to the inputs, save as `mux_sch.vwf`, then run Compilation and Simulation.
8. Record the outputs, then repeat for each input combination in Table **??**, saving the waveform file each time.
9. Print and submit the schematic diagram and simulation result.

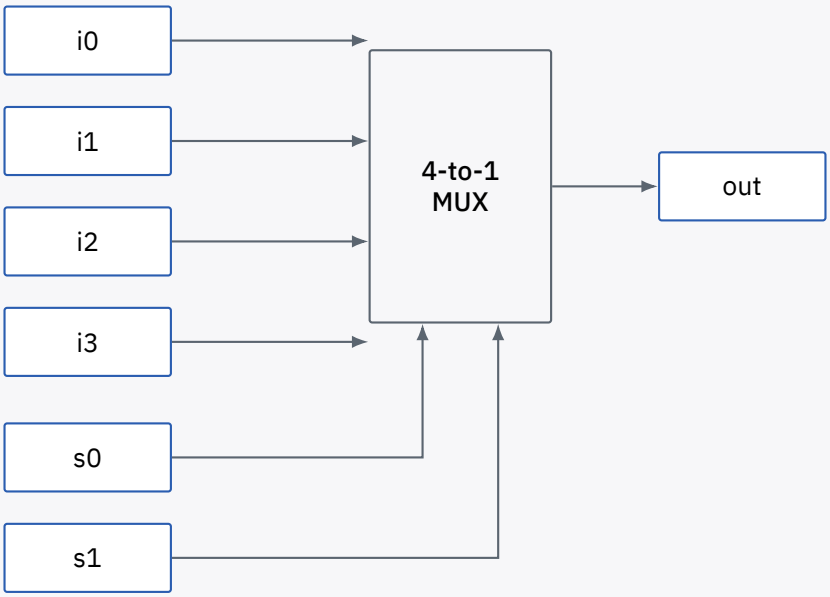


Figure 3: Schematic-entry 4-to-1 multiplexer (`mux_sch`).

S0	S1	I0	I1	I2	I3
0	0	1	1	0	0
0	1	0	1	1	0
1	0	1	1	0	1
1	1	1	1	0	0

Table 5: Input combinations to simulate.

Activity 2 — Verilog entry

Objective: use Verilog code as a design entry method in Quartus II.

1. Start a new project (`Activity2`, top level `mux_ver`) with the same device and EDA tool settings as Activity 1.
2. Create a new Verilog file and enter the code in Listing **??**.
3. Save as `mux_ver`, then compile the design.
4. Create a Vector Waveform File; insert input nodes `i0`–`i3`, `s0`, `s1`, and output node `out`.
5. Assign logic values matching Table **??**, save as `mux_ver.vwf`, then start simulation.
6. Inspect the result in the Simulation Report — Simulation Waveform.
7. Change the input values (saving before each run) and re-simulate.
8. View the synthesized schematic via **Tools** → **Netlist Viewer** → **RTL Viewer**, shown in Figure **??**.
9. Print and submit the Verilog code and simulation result.

mux_ver.v

```
module mux_ver(out, i0, i1, i2, i3, s1, s0);
  output out;
  input i0, i1, i2, i3;
  input s1, s0;
  reg out;

  always @(s1 or s0 or i0 or i1 or i2 or i3)
  begin
    case ({s1, s0})
      2'b00: out = i0;
      2'b01: out = i1;
      2'b10: out = i2;
      2'b11: out = i3;
      default: out = 1'bx;
    endcase
  end
endmodule
```

Listing 1: `mux_ver.v` — Verilog 4-to-1 multiplexer.

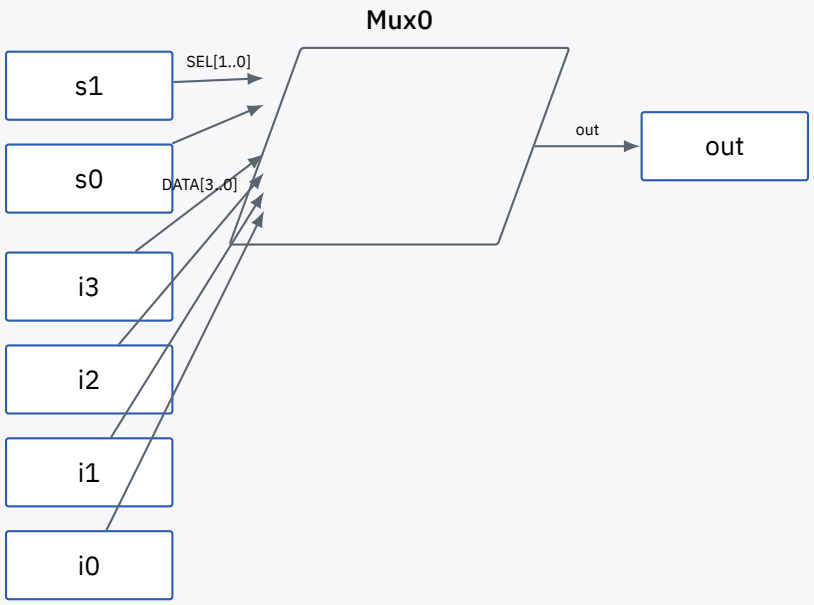


Figure 4: RTL Viewer output for `mux_ver` (`Mux0`).

Activity 3 — Combining schematic and Verilog entry

Objective: combine a Verilog module with schematic entry in a single design.

1. Start a new project (`activity3`, top level `mux_schver`) with the same device and EDA tool settings.
2. Create a new Verilog HDL file with the code in Listing **??**; save as `mux_vrl`.
3. Generate a reusable symbol from the file via **File → Create/Update → Create Symbol Files for Current File**. The resulting block is shown in Figure **??**.
4. Create a new Block Diagram/Schematic file, place the `mux_vrl` symbol from the project library, and connect its inputs and output to input/output pads as shown in Figure **??**.
5. Save as `mux_schver.bdf` and start compilation.
6. Create a Vector Waveform File, insert the same nodes as before, assign logic values, save as `mux_schver.vwf`, and run Compilation and Simulation.

7. Record the outputs for the combinations in Table **??**, saving the waveform file each time.
8. Print and submit the schematic diagram, Verilog code, and simulation result.

mux_vrl.v

```
module mux_vrl(out, i0, i1, i2, i3, s1, s0);
  output out;
  input i0, i1, i2, i3;
  input s1, s0;
  reg out;

  always @(s1 or s0 or i0 or i1 or i2 or i3)
  begin
    case ({s1, s0})
      2'b00: out = i0;
      2'b01: out = i1;
      2'b10: out = i2;
      2'b11: out = i3;
      default: out = 1'bx;
    endcase
  end
endmodule
```

Listing 2: `mux_vrl.v` — Verilog module to be reused as a schematic symbol.

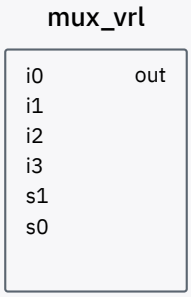


Figure 5: Auto-generated `mux_vrl` symbol block.

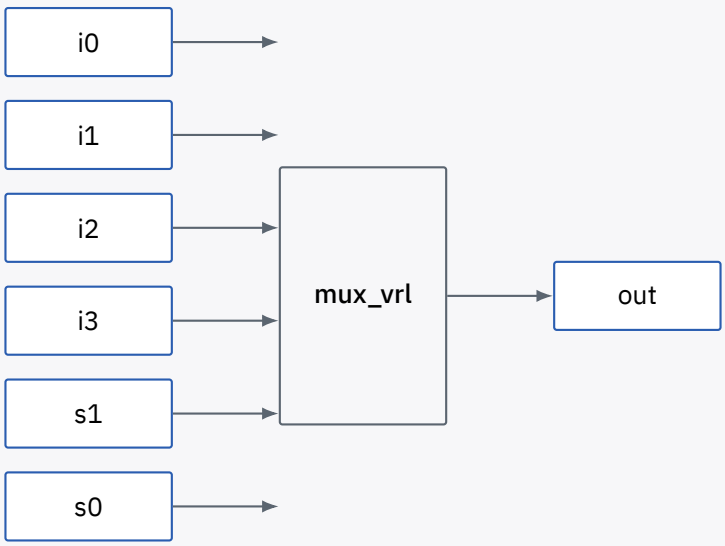


Figure 6: Combined schematic `mux_schver`: the Verilog-derived `mux_vrl` block wired to I/O pads.

Activity 4 — Timing analysis

Objective: analyse the maximum operating speed of a design.

1. Start a new project (`Activity4`, top level `counter`) with the same device and EDA tool settings.
2. Create a new Verilog file with the code in Listing **??**; save as `counter` and compile.
3. Create a Vector Waveform File; insert `clock` and `clear` as inputs and `Q` as output.
4. Force `clear` low, assign a clock signal to `clock`, save as `counter.vwf`, then start simulation.
5. Inspect the result in the Simulation Report — Simulation Waveform.
6. Run **Processing** → **Start** → **Start Classic Timing Analyzer**.
7. Read the maximum clock frequency from the Timing Analyzer Summary (Table **??**).
8. Print and submit the Verilog code and simulation result.

counter.v

```
module counter(Q, clock, clear);
  output [3:0] Q;
  input clock, clear;
  reg [3:0] Q;

  always @(posedge clear or negedge clock)
  begin
    if (clear)
      Q = 4'd0;
    else
      Q = (Q + 1) % 16;
  end
endmodule
```

Listing 3: `counter.v` — 4-bit up counter with asynchronous clear.

#	Type	Actual time	Failed paths
1	Worst-case t_{co} (clock → Q[3])	7.410 ns	0
2	Clock setup (`clock`)	233.64 MHz (4.280 ns period)	0
3	Total failed paths	—	0

Table 6: Timing Analyzer Summary for `counter`.

Activity 5 — Downloading to the DE2 board

Objective: download a Verilog design onto the Altera DE2 board and drive the seven-segment display.

1. Start a new project (`activity5`, top level `counta`) with the same device and EDA tool settings.
2. Create a new Verilog HDL file with the code in Listing **??**; save as `counta`.
3. Compile and simulate the design.
4. Assign each output pin to a physical device pin using **Assignment** → **Pin Planner**, per Table **??** (refer to the DE2 user manual for pin locations).
5. Save, then recompile the project.
6. Download the configuration to the DE2 board via **Tools** → **Programmer** → **Hardware Setup** → USB-Blaster → **Start**.
7. Record the pattern displayed on the seven-segment display.
8. Change the assignment to `assign q[6:0] = 7'b1111000;`, save, recompile, and download again. Record the new displayed pattern.

counta.v

```
module counta(q);
  output [6:0] q;
  assign q[6:0] = 7'b0010000;
endmodule
```

Listing 4: `counta.v` — static seven-segment pattern.

#	Node name	Direction	Pin
1	Q[6]	Output	PIN_V13
2	Q[5]	Output	PIN_V14
3	Q[4]	Output	PIN_AE11
4	Q[3]	Output	PIN_AD11
5	Q[2]	Output	PIN_AC12
6	Q[1]	Output	PIN_AB12
7	Q[0]	Output	PIN_AF10

Table 7: Seven-segment output pin assignment (DE2 board).

EEM 441 --- Control, Instrumentation & FPGA Laboratory

Experiment 4

Interfacing FPGA to DIP Switch and Seven Segment

Objectives

1. To use Verilog code as a design entry method in Altera Quartus II Software.
2. To interface a DIP switch and a seven-segment display to the FPGA.

Equipment / Software Required

- Altera Quartus II software
- Altera DE2 board
- Function generator

Introduction

Seven-segment displays are widely used to show numbers. The DE2 board provides eight seven-segment displays, 18 DIP switches, four push-button switches, and 16 LEDs. These components are hardwired on the board and are each assigned to specific FPGA pins — refer to the DE2 user manual for the full pin map.

Procedure

Activity 1 — 16-bit binary counter with frequency divider

Objective: design a 16-bit binary counter driven through a frequency divider.

1. Open Quartus II and write the frequency-divider module in Listing **??** (`seven`), which lowers the incoming clock frequency.
2. Create a reusable symbol for `seven`.
3. Write the 4-bit counter module in Listing **??** (`led`).
4. Create a reusable symbol for `led`.
5. Draw the schematic in Figure **??**, connecting the `seven` block's most significant output bit `q[16]` to the clock input of the `led` block.
6. Assign pin numbers as shown in Table **??** (refer to the DE2 user manual).
7. Use a function generator to supply the clock signal, using 1 MHz as the clock input.
8. Determine the effective input frequency reaching the `led` module.
9. Discuss your result based on the observed LED output display.

seven.v --- frequency divider

```
module seven(q, clk);
  output [15:0] q;
  input clk;
  reg [15:0] q;

  always @(posedge clk)
  begin
    q = (q + 1) % 65536;
  end
endmodule
```

Listing 5: `seven.v` — 16-bit frequency divider.

led.v --- 4-bit counter

```
module led(x, clk);
  input clk;
  output [3:0] x;
  reg [3:0] x;

  always @(posedge clk)
  begin
    x = (x + 1) % 16;
  end
endmodule
```

Listing 6: `led.v` — 4-bit binary counter.

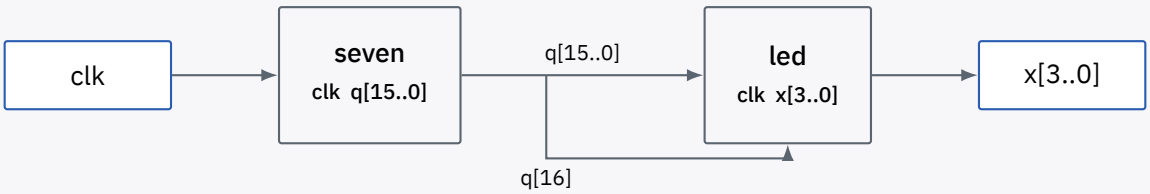


Figure 7: Schematic diagram for the 16-bit binary counter with frequency divider.

#	Node name	Direction	Pin location
1	clk	Input	PIN_D25
2	X[3]	Output	PIN_AC22
3	X[2]	Output	PIN_AB21
4	X[1]	Output	PIN_AF23
5	X[0]	Output	PIN_AE23

Table 8: Pin assignment for the 4-bit binary counter.

Activity 2 — 0–99 counter on seven-segment display

Objectives: (1) design a counter that counts 0 to 99; (2) display the result on the seven-segment displays.

1. Using a DIP switch as the trigger input, write Verilog code for a counter that counts from 0 to 99 once the switch is triggered.
2. Decode and display the result (tens and units digits) on two seven-segment displays.
3. Print and submit the Verilog code and simulation result.

Activity 3 — Running light

Objective: design a running-light display on the Altera DE2 board.

1. Using a DIP switch as the input, write Verilog code to implement a running-light (chasing LED) pattern.
2. Use three DIP switches to select between three different running-light patterns.
3. Print and submit the Verilog code and simulation result.

EEM 441 --- Control, Instrumentation & FPGA Laboratory

Experiment 5

Interfacing LCD to Altera DE2 Board

Objectives

1. To use Verilog code as a design entry method in Altera Quartus II Software.
2. To interface an LCD module to the Altera DE2 board.

Equipment / Software Required

- Altera Quartus II software
- Altera DE2 board

Introduction

In recent years, LCDs have largely replaced simple LED displays: they can show numbers, characters, and graphics, and are relatively easy to program. Table **??** lists the LCD pins relevant to programming.

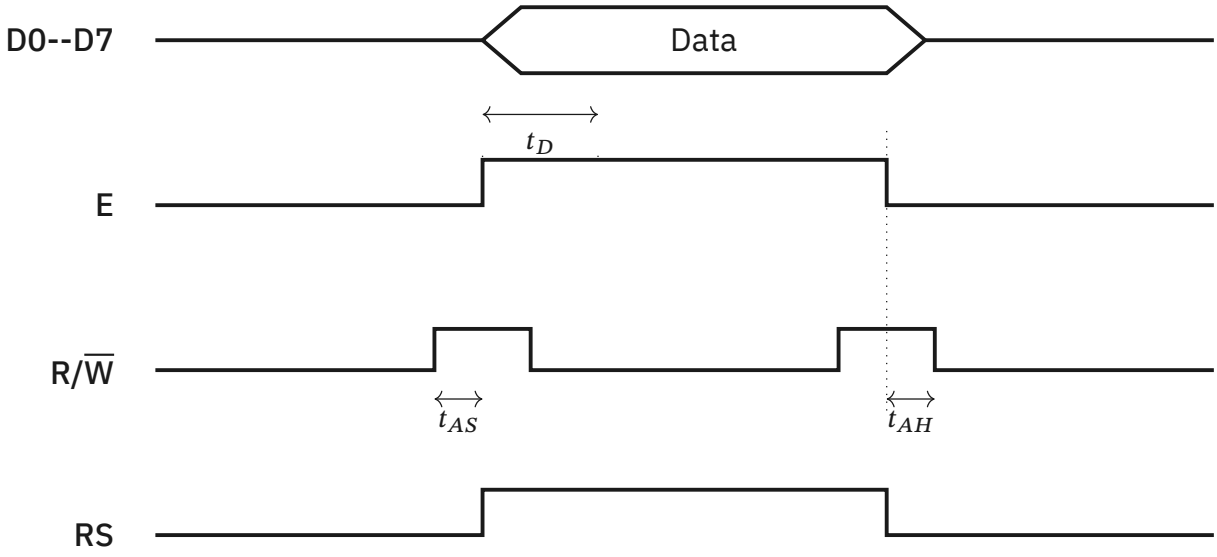
The *enable* (E) pin is used by the LCD to latch the information presented on its data pins: a high-to-low transition on E must be applied for the LCD to latch the data, and this pulse must be at least 450 ns wide. The required timing is shown in Figure **??**.

Two important internal registers are selected via the RS pin. When RS $= 0$, the *instruction/command* register is selected, allowing commands to be sent to the LCD (Table **??** lists the available command codes). When RS $= 1$, the *data* register is selected, allowing ASCII character data to be written for display, sent through data pins D0–D7.

It is good practice to check the busy flag before writing to the LCD: with RS $= 0$ and R/$\overline{\text{W}}$ $= 1$, the busy flag appears on D7. While D7 = 1 the LCD is busy with internal operations and will not accept new data; once D7 = 0, the LCD is ready to receive the next byte.

Pin	Symbol	I/O	Description
1	V_{ss}	—	Ground
2	V_{cc}	—	+5 V power supply
3	V_{ee}	—	Power supply to control contrast
4	RS	I	RS $= 0$: command register; RS $= 1$: data register
5	R/$\overline{\text{W}}$	I	0: write; 1: read
6	E	I/O	Enable (latches data on high-to-low transition)
7	DB0	I/O	8-bit data bus
8	DB1	I/O	
9	DB2	I/O	
10	DB3	I/O	
11	DB4	I/O	
12	DB5	I/O	
13	DB6	I/O	
14	DB7	I/O	

Table 9: Pin description for the LCD module.



t_D = data output delay time
t_{AS} = setup time prior to E (going high) for RS and R/$\overline{W}$, minimum 140 ns
t_{AH} = hold time after E falls, for RS and R/$\overline{W}$, minimum 10 ns
Note: a read operation requires an L-to-H pulse on the E pin.

Figure 8: LCD write-cycle timing diagram.

Code (hex)	Command to LCD instruction register
1	Clear display screen
2	Return home
4	Decrement cursor (shift cursor left)
6	Increment cursor (shift cursor right)
5	Shift display right
7	Shift display left
8	Display off, cursor off
A	Display off, cursor on
C	Display on, cursor off
E	Display on, cursor blinking
F	Display on, cursor blinking
10	Shift cursor position left
14	Shift cursor position right
18	Shift entire display left
1C	Shift entire display right
80	Force cursor to beginning of line 1
C0	Force cursor to beginning of line 2
38	2 lines, 5×7 dot matrix

Table 10: LCD command codes.

Procedure

Activity 1 — Displaying lookup-table data on the LCD

Objective: display fixed text, stored in a lookup table, on the LCD.

1. Open Quartus II.

2. Write the Verilog modules given in Listing **??** (`lcd_t`) and Listing **??** (`LCD_Controller`) to send commands and data to the LCD.
3. `lcd_t` uses a lookup table (LUT) to store the sequence of commands and characters to send to the LCD.
4. Create a reusable symbol for each module, as in Experiment 3.
5. Draw the schematic in Figure **??**, connecting `lcd_t`'s outputs to `LCD_Controller`'s inputs, and `LCD_Controller`'s outputs to the LCD interface pins.
6. Assign pin numbers as shown in Table **??**.
7. Download the program to the Altera DE2 board.
8. Discuss the result based on your observation of the LCD.

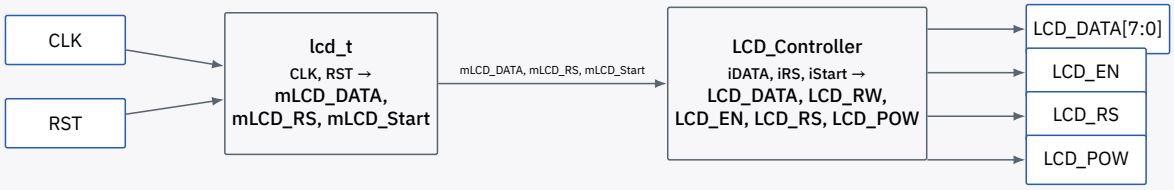


Figure 9: Schematic diagram for the LCD interface: `lcd_t` (lookup-table sequencer) feeding `LCD_Controller` (bus-timing controller).

#	Node name	Direction	Location
1	LCD_DATA[7]	Output	PIN_H3
2	LCD_DATA[6]	Output	PIN_H4
3	LCD_DATA[5]	Output	PIN_J3
4	LCD_DATA[4]	Output	PIN_J4
5	LCD_DATA[3]	Output	PIN_H2
6	LCD_DATA[2]	Output	PIN_H1
7	LCD_DATA[1]	Output	PIN_J2
8	LCD_DATA[0]	Output	PIN_J1
9	LCD_EN	Output	PIN_K3
10	LCD_POW	Output	PIN_L4
11	LCD_RS	Output	PIN_K1
12	LCD_RW	Output	PIN_K4
13	CLK	Input	PIN_N2
14	RST	Input	PIN_G26

Table 11: Pin assignment for the LCD interface.

lcd_t.v --- Appendix 1: lookup-table sequencer

```
module lcd_t (
    // Host side
    CLK, RST, mLCD_Done,
    mLCD_DATA, mLCD_RS, mLCD_Start
);
  // Host side
  input  CLK, RST, mLCD_Done;
  // Internal wires/registers
  output [7:0] mLCD_DATA;
  output mLCD_RS, mLCD_Start;
  reg  [5:0] LUT_INDEX;
  reg  [8:0] LUT_DATA;
  reg  [5:0] mLCD_ST;
  reg  [17:0] mDLY;
  reg  mLCD_Start;
  reg  [7:0] mLCD_DATA;
  reg  mLCD_RS;
  wire mLCD_Done;

  parameter LCD_INITIAL = 0;
  parameter LCD_LINE1   = 5;
  parameter LCD_CH_LINE = LCD_LINE1 + 16;
  parameter LCD_LINE2   = LCD_LINE1 + 16 + 1;
  parameter LUT_SIZE    = LCD_LINE1 + 32 + 1;

  // Sequencing state machine: send one LUT entry, wait for
  // LCD_Controller to finish (mLCD_Done), pause, then advance.
  always @(posedge CLK or negedge RST)
  begin
    if (!RST)
    begin
      LUT_INDEX  <= 0;
      mLCD_ST    <= 0;
      mDLY       <= 0;
      mLCD_Start <= 0;
      mLCD_DATA  <= 0;
      mLCD_RS    <= 0;
    end
    else if (LUT_INDEX < LUT_SIZE)
    begin
      case (mLCD_ST)
        0: begin
          mLCD_DATA  <= LUT_DATA[7:0];
          mLCD_RS    <= LUT_DATA[8];
          mLCD_Start <= 1;
          mLCD_ST    <= 1;
        end
        1: if (mLCD_Done) begin
          mLCD_Start <= 0;
          mLCD_ST    <= 2;
        end
        2: if (mDLY < 18'h3FFFE)
             mDLY <= mDLY + 1;
           else begin
             mDLY    <= 0;
             mLCD_ST <= 3;
           end
        3: begin
          LUT_INDEX <= LUT_INDEX + 1;
          mLCD_ST   <= 0;
        end
```

Listing 7: `lcd_t.v` — lookup-table LCD message sequencer.

LCD_Controller.v --- Appendix 2: bus-timing controller

```
module LCD_Controller (
    // Host side
    iDATA, iRS, iStart, oDone, CLK, RST,
    // LCD interface
    LCD_DATA, LCD_RW, LCD_EN, LCD_RS, LCD_POW
);
  parameter CLK_Divide = 16;

  // Host side
  input  [7:0] iDATA;
  input  iRS, iStart;
  input  CLK, RST;
  output reg oDone;

  // LCD interface
  output [7:0] LCD_DATA;
  output reg LCD_EN;
  output LCD_RW;
  output LCD_RS;
  output LCD_POW;

  // Internal registers
  reg [4:0] Cont;
  reg [1:0] ST;
  reg preStart, mStart;

  // Write-only interface: bypass iRS straight to LCD_RS
  assign LCD_DATA = iDATA;
  assign LCD_RW   = 1'b0;
  assign LCD_RS   = iRS;
  assign LCD_POW  = 1'b1;

  always @(posedge CLK or negedge RST)
  begin
    if (!RST)
    begin
      oDone    <= 1'b0;
      LCD_EN   <= 1'b0;
      preStart <= 1'b0;
      mStart   <= 1'b0;
      Cont     <= 0;
      ST       <= 0;
    end
    else
    begin
      // Rising-edge detect on iStart
      preStart <= iStart;
      if ({preStart, iStart} == 2'b01)
      begin
        mStart <= 1'b1;
        oDone  <= 1'b0;
      end

      if (mStart)
      begin
        case (ST)
          0: ST <= 1;                 // wait setup
          1: begin
            LCD_EN <= 1'b1;           // raise enable
            ST     <= 2;
          end
```

Listing 8: `LCD_Controller.v` — generates the LCD enable-pulse timing.

Activity 2 — Displaying DIP switch input on the LCD

Objective: display the live 8-bit DIP switch configuration on the LCD.

1. Write Verilog code that reads the 8-bit DIP switch configuration and displays it on the LCD (e.g. if switch 1 is turned on, the LCD should display 1 in the corresponding position).
2. Print and submit the Verilog code and simulation result.

EEM 441 --- Control, Instrumentation & FPGA Laboratory

Experiment 6

Stepper Motor Control with FPGA

Objectives

1. To design digital circuits in an FPGA to control a stepper motor.
2. To use Verilog as a design entry method in Altera Quartus II Software.
3. To simulate the circuit using Altera Quartus II Software.
4. To download the design onto an Altera DE2 board.

Equipment / Software Required

- Altera Quartus II software
- Altera DE2 board
- Parallax 4-phase, 12 V, 75 Ω unipolar stepper motor
- ULN2003 Darlington transistor array (motor driver)

Introduction

Unlike an ordinary DC motor, a stepper motor does not spin freely when power is applied — it must be driven with a continuously pulsed sequence of specific coil-energising patterns. Each pulse advances the rotor by a fixed step angle (for this motor, $3.6°$ per step), which is what gives stepper motors their positioning precision: as long as speed and torque stay within the motor's limits, the controller always knows the rotor's exact position.

The rotor is driven by the interaction of magnetic fields as coils are switched on and off in sequence. Table **??** shows the normal four-coil unipolar step sequence; taking a step means driving two of the four coil-driver pins high at a time, in the order shown, through the ULN2003 driver.

	Step 1	Step 2	Step 3	Step 4	Step 1
Coil 1 (A)	1	1	0	0	1
Coil 2 ($\bar{A}$)	0	0	1	1	0
Coil 3 (B)	1	0	0	1	1
Coil 4 ($\bar{B}$)	0	1	1	0	0

Table 12: Normal step sequence for a four-coil unipolar stepper motor.

Procedure

Activity 1 — Verilog stepper motor controller

Objective: write Verilog code to control the motion of a stepper motor.

1. Open Quartus II and start the New Project Wizard.
   - Working directory: `fpga`; project name: `activity1`; top-level entity: `step`.

- Device family: Cyclone II, device `EP2C35F672C6N`.
- Accept the default EDA tool settings and finish.

2. Create a new Verilog HDL file and enter the code in Listing **??** (`step`, `divider`, and `stepper` modules).
3. Save as `step` and start compilation.
4. Use **Tools** → **Netlist Viewer** → **RTL Viewer** to generate the block overview shown in Figure **??**.
5. Assign pin numbers as shown in Table **??** (refer to the DE2 user manual). For the `phaseout` outputs that interface with the stepper motor driver, use expansion header `GPIO_0`.
6. Download the program to the Altera DE2 board.
7. Discuss the result.

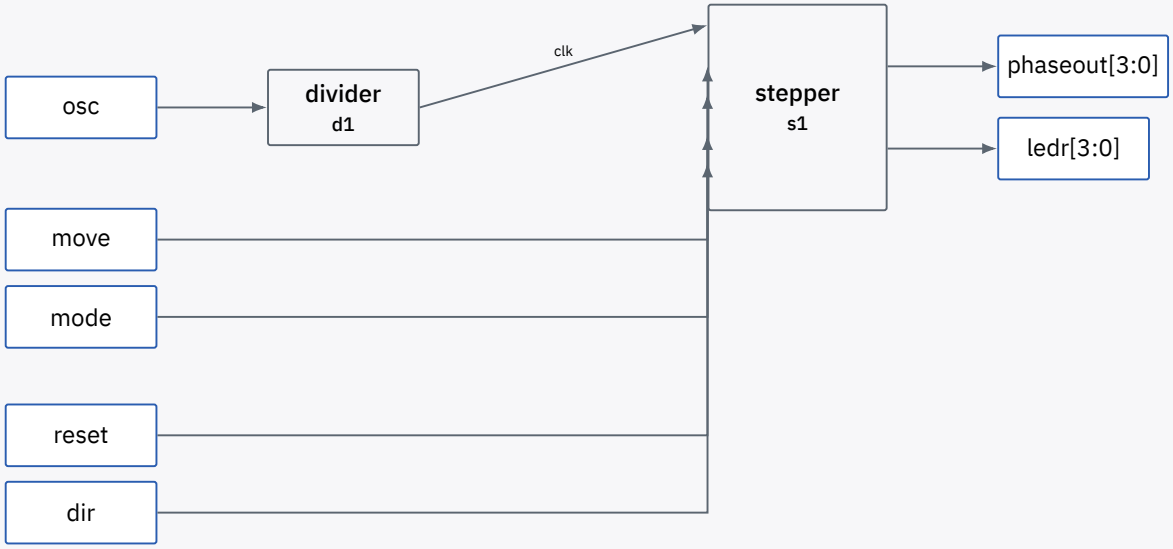


Figure 10: RTL Viewer overview of the `step` top-level module.

#	Node name	Direction	Pin location
1	osc	Input	PIN_D25
	dir (SW0)	Input	PIN_N25
	mode (SW1)	Input	PIN_N26
	move (SW2)	Input	PIN_P25
	reset (SW3)	Input	PIN_AE14
2	ledr[3]	Output	PIN_AC22
3	ledr[2]	Output	PIN_AB21
4	ledr[1]	Output	PIN_AF23
5	ledr[0]	Output	PIN_AE23
6	phaseout[3]	Output	PIN_E25
7	phaseout[2]	Output	PIN_E26
8	phaseout[1]	Output	PIN_J22
9	phaseout[0]	Output	PIN_D25

Table 13: Pin assignment for stepper motor control.

step.v --- top-level module

```
//============================================================
// Stepper Motor Controller
//============================================================
// TOP LEVEL MODULE
// reset (active low) brings the shaft to its reference position.
// mode selects step (1) or continuous (0) operation.
// dir selects clockwise (1) or anti-clockwise (0) motion.
// move shifts the shaft by one half-step in step mode.
module step(osc, reset, dir, mode, move, phaseout, ledr);
  input  osc, reset, mode, dir, move;
  output [3:0] phaseout, ledr;

  wire clk;

  divider d1 (
    .osc (osc),
    .clk (clk)
  );

  stepper s1 (
    .clk      (clk),
    .reset    (reset),
    .dir      (dir),
    .mode     (mode),
    .move     (move),
    .phaseout (phaseout)
  );
endmodule
```

step.v --- motor controller module (excerpt)

```
// Module stepper is the motor controller.
module stepper(clk, reset, dir, mode, move, phaseout);
  input  clk, reset, dir, mode, move;
  output [3:0] phaseout;
  reg [3:0] phaseout, position, ledr;

  always @(posedge clk or negedge reset)
  begin
    if (reset == 0) begin
      // motor resets to phase 1000 when reset is asserted
      phaseout = 4'b1000;
      ledr     = phaseout;
    end
    else begin
      if (mode == 0) begin
        // Mode = 0: continuous motion.
        // The 8-state sequence below follows Table 1 in both
        // directions; only the first two states are shown here,
        // the remaining six states (0100, 0110, 0010, 0011,
        // 0001, 1001) follow the identical if(dir==0)/else
        // pattern, cycling back to 1000.
        case (phaseout)
          4'b1000: if (dir == 0) begin
            phaseout = 4'b1100;  ledr = phaseout;
          end else begin
            phaseout = 4'b1001;  ledr = phaseout;
          end
          4'b1100: if (dir == 0) begin
            phaseout = 4'b0100;  ledr = phaseout;
          end else begin
            phaseout = 4'b1000;  ledr = phaseout;
          end
          // ... 4'b0100, 4'b0110, 4'b0010, 4'b0011,
          //     4'b0001, 4'b1001 follow the same pattern ...
          default: $display("invalid input");
        endcase
      end
      else begin
        // Mode = 1: single-step motion, advances only when
        // move == 0. Uses the same 8-state sequence as above,
        // indexed by `position` instead of `phaseout`.
        if (move == 1'b0) begin
          case (position)
            // ... identical 8-state sequence as continuous
            //     mode, see Table 1 ...
            default: $display("invalid input");
          endcase
        end
      end
    end
  end

  // Motor advances by one half-step on the falling edge of move.
  always @(negedge move or negedge reset)
  begin
    if (reset == 0) begin
      position = 4'b1000;  // NOTE: corrected from the original
                           // listing, which printed this value
                           // as "4.101011" -- likely a
                           // transcription error; 4'b1000
                           // matches the reset state used
```

step.v --- clock divider module

```
// Module divider divides the incoming clock to produce a
// clock suitable for driving the motor controller.
module divider(osc, clk);
  input  osc;
  output clk;
  reg    clk;
  reg [15:0] count;

  initial count = 16'b0000000000000000;

  always @(posedge osc)
  begin
    count = count + 1;
    clk   = count[15];
  end
endmodule
```

Listing 9: `step.v` — top-level, motor-controller, and clock-divider modules.

Pending

The controller module contains a repetitive 8-state case statement (one branch per motor phase, in both rotation directions) that has been abbreviated above for readability. The full state sequence is fully specified by Table **??**: each state transitions to the next (or previous) entry in the cycle depending on `dir`, wrapping around after the eighth state.

Activity 2 — Variable-speed control with display feedback

Objective: extend the controller with selectable speed and direction/mode feedback.

1. Write Verilog code that allows selection of three different speeds for the stepper motor in continuous-motion mode.
2. Display the motor's direction of rotation on the seven-segment display, and the current mode (step / continuous) on the LCD display.
3. Print and submit the Verilog code and schematic diagram.

References

1. Altera DE2 User Manual.
2. *Introduction to Quartus II Software — Verilog, VHDL, Schematic*, Altera University Program.
3. *Introduction to Verilog*, Carleton University course notes.
4. *Stepper Motor Controller Using MAX II CPLDs*, Altera Application Note AN-488.
5. Parallax Inc., *4-Phase 12-Volt 75 Ω Unipolar Stepper Motor (#27964)*, technical datasheet.

EEM 441 --- Control, Instrumentation & FPGA Laboratory

Experiment 7

Instrumentation --- Measuring the Speed of a DC Motor Using a Computer

Objectives

1. To understand the methods of acquiring measurements automatically with the aid of a computer.
2. To become familiar with the data-acquisition (DAQ) environment.
3. To become familiar with signal conditioning in a computer-based data-acquisition system.
4. To design and build a signal-conditioning circuit whose parameters are justified by a measurement, rather than assumed.
5. To implement a computer-based measurement system for the rotational speed of a DC motor.

Equipment / Software Required

- DC motor trainer with a 12-slot wheel encoder
- Digital storage oscilloscope (DSO)
- Components for the student-designed filter and signal-conditioning circuit (op-amps, resistors, capacitors)
- ADAM-3968 wiring terminal board
- PCIE-1816 multifunction DAQ card, installed in the host PC
- PC with a C/C++ or Python development environment

Introduction

In most measurement processes it is often necessary to acquire data automatically, without a human operator. A computer-based instrument has several advantages over conventional measuring instruments: it can acquire measurements quickly and accurately, sustain real-time control, and operate for extended periods with consistent precision.

A computer-based measurement system typically contains the following elements (Figure **??**): a transducer, a signal-conditioning stage, data-acquisition hardware, analysis hardware, and software running on the host PC.

Signal conditioning means manipulating an analogue signal so that it meets the requirements of the next processing stage — most commonly, so that it is suitable for an analogue-to-digital converter. Signal conditioning can include amplification, filtering, range matching, and isolation.

Filtering is the most common signal-conditioning function: frequencies that do not carry valid data are attenuated. In this experiment the encoder produces a pulse train whose useful frequency content is bounded, and any filter used ahead of the DAQ card should be designed — not assumed — around that bound.

Amplification increases the resolution of the input signal and improves its signal-to-noise

ratio.
Isolation allows a signal to pass from its source to the measurement device without a direct physical (galvanic) connection, protecting sensitive equipment from the signal source and vice versa.
The Sallen–Key topology is a widely used active-filter structure, valued for its simplicity: it realises a two-pole (12 dB/octave) low-pass, high-pass, or band-pass response using a single op-amp configured as a near-unity-gain voltage-controlled voltage source (VCVS), together with two resistors and two capacitors that set the cutoff frequency and Q.

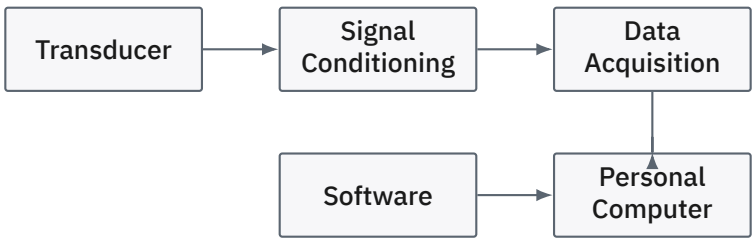


Figure 11: Computer-based measurement system: transducer, signal conditioning, DAQ hardware, and analysis software.

Procedure

Activity 1 — Characterising the encoder signal and designing the filter

Objective: measure the encoder's pulse characteristics and design a signal-conditioning filter around the measured cutoff frequency, rather than an assumed one.

1. Wire the DC motor trainer so that the 12-slot wheel encoder's output is available at a test point, and connect this test point to the oscilloscope.
2. Run the motor across its usable speed range and observe the encoder pulse train on the oscilloscope. Each revolution produces 12 pulses (one per slot).
3. At a representative operating speed, measure the pulse period and estimate the corresponding fundamental frequency and its dominant harmonics directly from the oscilloscope trace.
4. From this measurement, determine a suitable cutoff frequency for an anti-aliasing / noise-rejection low-pass filter: the cutoff must pass the pulse's fundamental frequency (across the motor's full speed range) while rejecting high-frequency switching noise.
5. **Design task:** using the Sallen–Key topology introduced above, design a second-order active low-pass filter with the cutoff frequency determined in step 4. Select a topology (unity-gain or gain-set), calculate resistor and capacitor values, and justify your component choices in your report.
6. Build and test the filter on its own (e.g. with a function generator) to verify its measured cutoff frequency and roll-off match your design before connecting it to the encoder signal.
7. **Design task:** design the remaining signal-conditioning stage needed to present the filtered encoder pulses to the PCIE-1816's digital input with valid logic levels (buffering, level-shifting, or pulse-shaping as required). Justify your design choices.

Pending

The filter and signal-conditioning circuit are student design deliverables: there is no fixed cutoff frequency or component value specified here. Document your oscilloscope measurements, design calculations, and any deviation between the designed and measured filter response in your report.

Activity 2 — Interfacing to the DAQ hardware

Objective: connect the conditioned encoder signal to the PC via the ADAM-3968 and PCIE-1816.

1. Connect the output of your signal-conditioning circuit to a digital input (or counter/timer input, if using the PCIE-1816's onboard counter) on the ADAM-3968 terminal board.
2. Connect the ADAM-3968 to the PCIE-1816 DAQ card installed in the host PC via the supplied cable.
3. Verify the wiring using the DAQ card's bundled configuration/monitoring utility before writing any code, to confirm the expected pulses are being registered.

Figure 12: Signal path from the DC motor trainer to the host PC.

Activity 3 — Speed measurement software

Objective: implement software to compute and display motor speed in real time.
The rotational speed can be found from the pulse period, using the number of pulses produced per revolution (12, for this encoder):

$$\text{Period} = \frac{t_{\text{stop}} - t_{\text{start}}}{\text{CLK\_TCK}}, \qquad \omega = \frac{2\pi}{12 \times \text{Period}} \ \text{[rad/s]}$$

where t_{start} and t_{stop} mark successive encoder edges and CLK_TCK is the counter's clock tick rate.

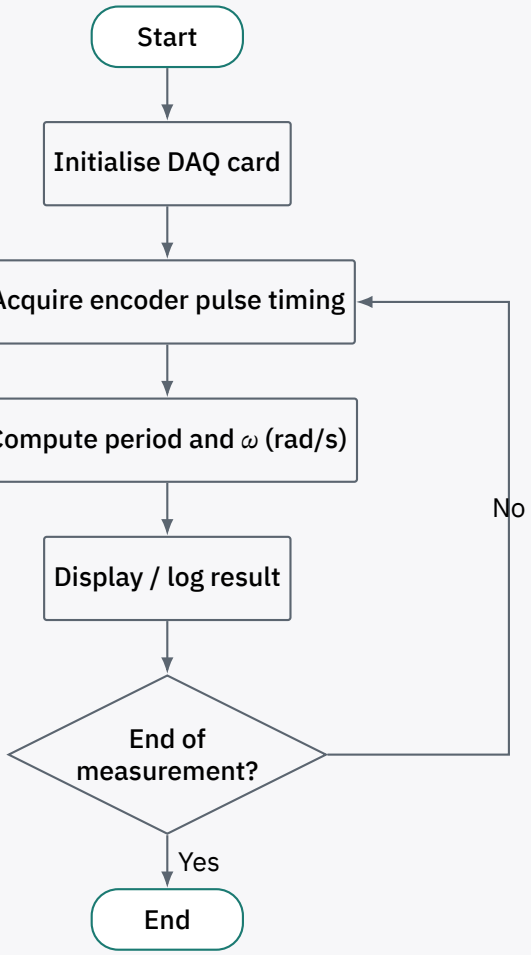


Figure 13: Measurement loop for the speed-acquisition software.

1. In C, C++, or Python, write a program that:
   - initialises the PCIE-1816 (via the vendor driver / SDK, or the DAQ card's counter API),
   - times successive encoder pulses (or reads the onboard counter) to obtain the pulse period,
   - computes the rotational speed in rad/s (or rpm) using the relation above, and
   - displays the running value in a simple GUI, updating in real time.
2. Interface your filter/conditioning circuit, the ADAM-3968, and the PCIE-1816 together and perform measurements across a range of motor input voltages. Compare your computed speed against a direct oscilloscope measurement of the encoder period at the same operating point.
3. Tabulate your results (Table **??**).
4. Plot computed speed (from your software) against the oscilloscope-measured speed.
5. Discuss and comment on your results, including any discrepancy between the two measurement methods and possible causes (filter phase delay, counting resolution, etc.).

	Input voltage		
	# 1	# 2	# 3
Speed (rad/s) — computer			
Speed (rad/s) — oscilloscope			

Table 14: Computed vs. oscilloscope-measured motor speed.

Report

1. Present your oscilloscope measurements of the encoder pulse and your resulting filter design (topology, calculations, and component values), together with the measured response of the built filter.
2. Present your design of the signal-conditioning stage between the filter and the PCIE-1816 digital input, with justification.
3. Compare your software-computed speed with the oscilloscope-measured speed across your tested operating points, and discuss the sources of any discrepancy.
4. What is the theoretical maximum measurable speed of this system, given the encoder resolution (12 slots/revolution) and your software's timing resolution?

EEM 441 --- Control, Instrumentation & FPGA Laboratory

Experiment 8

Instrumentation --- Temperature Measurement and Control

Objectives

1. To design and build a signal-conditioning circuit for an NTC thermistor.
2. To interface the conditioned temperature signal to the PC via the PCIE-1750-U and ADAM-3937.
3. To implement a computer-based system that displays the measured temperature and performs ON/OFF control of an indicator LED on the trainer.

Equipment / Software Required

- NTC thermistor and trainer board with indicator LED
- Components for the student-designed signal-conditioning circuit (op-amps, resistors, capacitors)
- ADAM-3937 DB37 wiring terminal board
- PCIE-1750-U isolated digital I/O card, installed in the host PC
- PC with a C/C++ or Python development environment
- Digital multimeter

Introduction

In many industrial processes, temperature must be regulated automatically. Two general strategies are used: *logic (ON/OFF) control*, in which a heater, cooler, or indicator is simply switched on or off when a threshold is crossed, and *fuzzy/proportional control*, which requires a more elaborate definition of “hot” or “cold” before acting. This experiment uses logic control.

An NTC (negative temperature coefficient) thermistor is a resistor whose resistance decreases as its temperature increases. Given the thermistor’s resistance–temperature characteristic (from its datasheet), a measured resistance can be converted to a temperature, and vice versa.

The signal path for this experiment is: the thermistor’s resistance is converted to a voltage by a signal-conditioning circuit (which the student must design), this voltage is digitised and read by the PC via the PCIE-1750-U (through the ADAM-3937 wiring terminal), and software on the PC computes the corresponding temperature, displays it, and drives the indicator LED on the trainer according to a threshold comparison (Figure **??**).

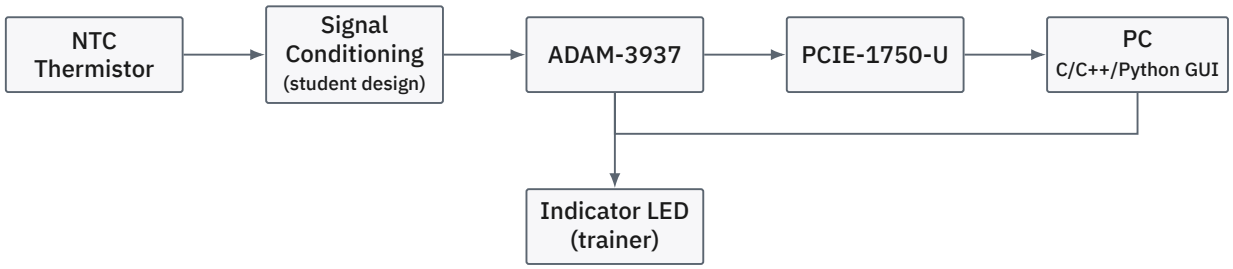


Figure 14: Block diagram of the temperature measurement and control system.

Procedure

Activity 1 — Signal-conditioning circuit design

Objective: design and build a circuit that converts the thermistor's resistance to a voltage suitable for the PCIE-1750-U input.

1. Obtain the resistance–temperature characteristic for the NTC thermistor from its datasheet, and determine the resistance range corresponding to the temperature range you intend to measure and control.
2. **Design task:** design a signal-conditioning circuit (e.g. a voltage divider or Wheatstone-bridge front end, followed by a buffer/amplifier stage) that converts the thermistor's resistance to a voltage. Calculate your component values and justify your design choices, including the expected output voltage range over your chosen temperature range.
3. **Design task:** include a voltage-follower (buffer) stage to eliminate loading effects between the conditioning circuit and the downstream measurement hardware.
4. Build the circuit and verify its output voltage against a reference temperature measurement (e.g. a calibrated thermometer or a known-temperature bath) at two or more points, and check linearity/consistency with your design calculations.

Pending

The signal-conditioning circuit is a student design deliverable: there is no fixed topology or component value specified here. Document your design calculations, the datasheet values used, and your calibration measurements in your report.

Activity 2 — Interfacing to the DAQ hardware

Objective: connect the conditioned temperature signal and the LED control line to the PC via the ADAM-3937 and PCIE-1750-U.

1. Connect the output of your signal-conditioning circuit to an input channel on the ADAM-3937 terminal board.
2. Connect the trainer's indicator LED control line to an isolated digital output channel on the ADAM-3937.
3. Connect the ADAM-3937 to the PCIE-1750-U card installed in the host PC via the supplied cable.
4. Verify both the input and output wiring using the DAQ card's bundled configuration/monitoring utility before writing any code: confirm the conditioned voltage reads correctly, and confirm the LED can be toggled from the utility.

Activity 3 — Temperature monitoring and LED control software

Objective: implement software that displays the measured temperature and controls the LED according to a threshold.

1. In C, C++, or Python, write a program that:

- initialises the PCIE-1750-U (via the vendor driver / SDK),
- periodically reads the conditioned thermistor voltage,
- converts the reading to a temperature using your circuit's calibration from Activity 1,
- displays the current temperature in a simple GUI, updating in real time, and
- compares the measured temperature against a user-specified threshold and sets the LED output accordingly.

2. **Design task:** decide, and clearly state and justify in your report, whether the LED should turn ON above or below the threshold (e.g. as a "too hot" alarm vs. a "still heating" indicator) — there is no single correct choice; your design should suit a stated purpose.
3. Use the flowchart in Figure **??** as a starting point for your control logic, adapting the ON/OFF branch to match your chosen design.

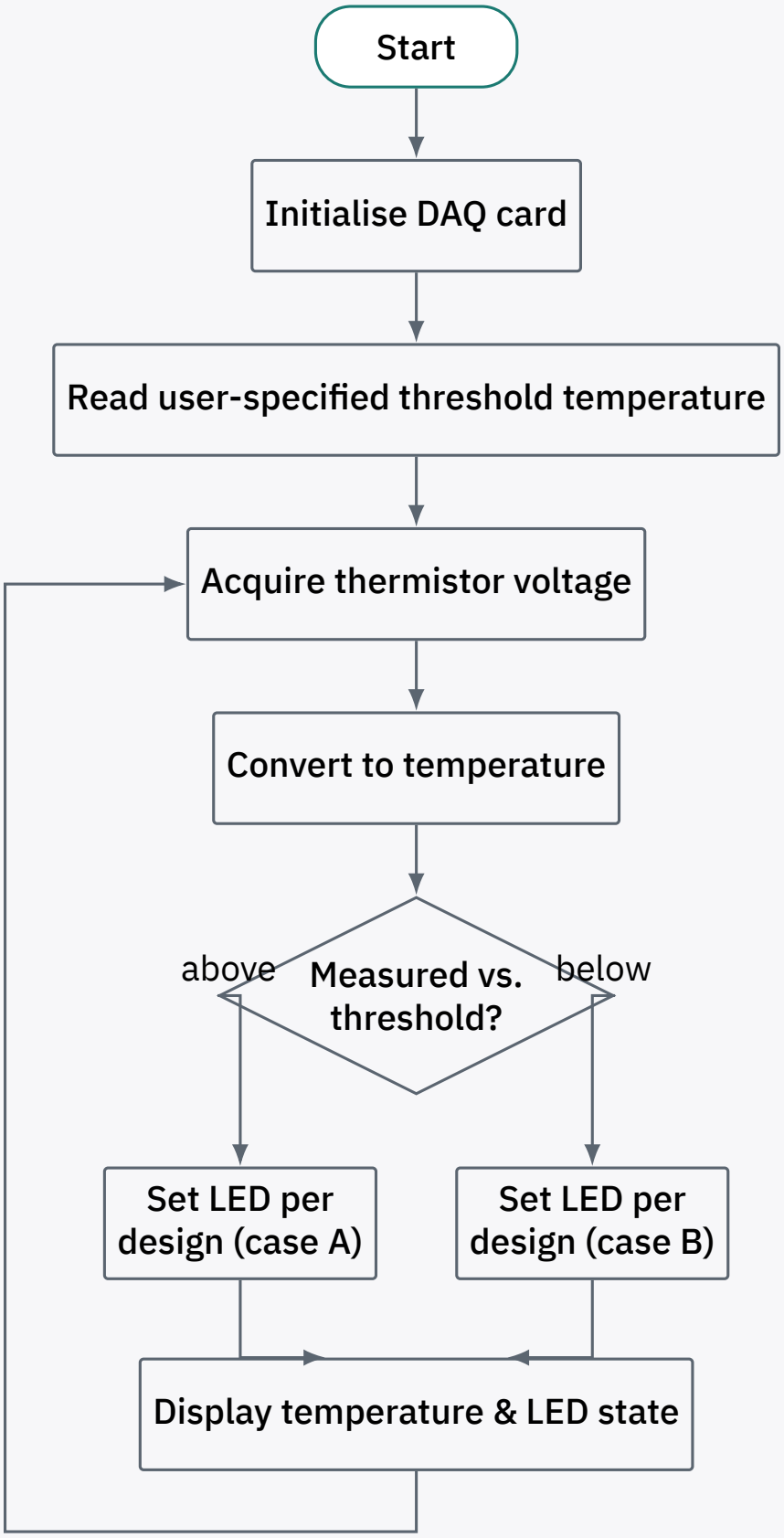


Figure 15: Temperature monitoring and LED control loop (ON/OFF direction per student design).

4. Set the controlled (threshold) temperature to a chosen value and record the measured temperature and LED state at 1-minute intervals for 35 minutes (Table **??**). Plot temperature against time.

5. Repeat for at least two further threshold temperatures. Plot your results.
6. Discuss your findings, including the responsiveness of the LED control and the accuracy of your temperature conversion.

Threshold temp.	Time (min)	Measured temp.	LED state
	1		
	2		
	...		
	35		

Table 15: Temperature and LED-state log.

Report

1. Present your signal-conditioning circuit design (topology, calculations, component values) and its calibration against a reference temperature measurement.
2. State and justify your chosen LED ON/OFF logic (Activity 3, step 2).
3. Present and discuss your temperature-vs-time plots for each threshold tested.
4. What sources of error or lag affect the accuracy and responsiveness of this system (thermistor thermal mass, signal-conditioning bandwidth, software polling rate, etc.)?

EEM 441 --- Control, Instrumentation & FPGA Laboratory

Experiment 9

Interfacing FPGA to an Analog-to-Digital Converter

Objectives

1. To understand the operation of an analog-to-digital converter (ADC).
2. To interface an FPGA, via the Altera DE2 board, to an ADC.

Equipment / Software Required

- Altera DE2 board
- Multimeter / oscilloscope
- ADC0804 analog-to-digital converter
- Resistors: $10\,\text{k}\Omega$
- Variable resistor (potentiometer), $10\,\text{k}\Omega$
- Capacitor: $150\,\text{pF}$
- 8 LEDs
- 74LS245 octal bus transceiver

Introduction

This experiment explores interfacing an ADC chip to an FPGA. ADCs are among the most widely used data-acquisition devices: they translate an analog signal into a digital number that a processor (or, here, an FPGA) can read. A widely used ADC is the ADC0804 (National Semiconductor): it operates from $+5$ V and has 8-bit resolution.
Besides resolution, *conversion time* is a major factor in judging an ADC: the time the ADC takes to convert its analog input to a digital number. For the ADC0804 this depends on the clock signal applied to the CLK R / CLK IN pins, but cannot be faster than $110\,\mu$s.
The ADC0804's input range is set by the $V_{ref}/2$ pin, as shown in Table **??**. The chip's outputs are 5 V logic; since the DE2 board's inputs only tolerate 3.3 V, an octal bus transceiver (74LS245) must be used to step the output voltage down before it reaches the FPGA.

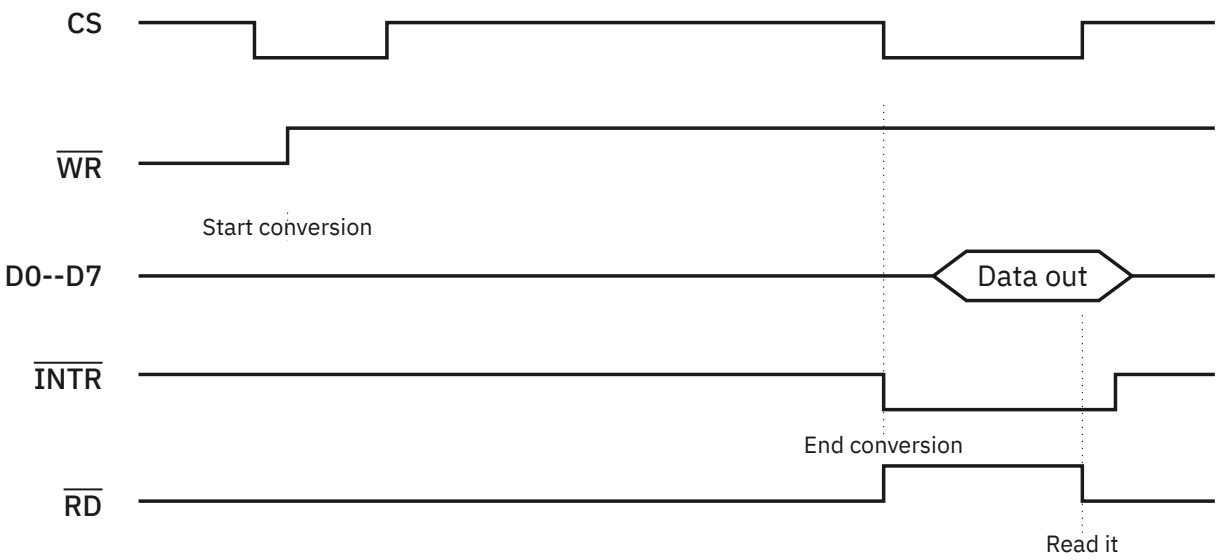
$$D_{out} = \frac{V_{in}}{\text{step size}}$$

where D_{out} is the digital data output and V_{in} is the analog input voltage.
Data conversion with the ADC0804 follows this sequence (Figure **??**):

i. Set $\overline{\text{CS}} = 0$ and send a low-to-high pulse on the $\overline{\text{WR}}$ pin to start the conversion.
ii. Monitor the $\overline{\text{INTR}}$ pin: while it is high, the conversion is still in progress; once it goes low, the conversion is complete.
iii. With $\overline{\text{INTR}}$ low, set $\overline{\text{CS}} = 0$ again and send a high-to-low pulse on the $\overline{\text{RD}}$ pin to read the data out of the ADC0804.

$V_{ref}/2$ (V)	V_{in} range (V)	Step size (mV)
Not connected	0 to 5	$5/256 = 19.53$
2.0	0 to 4	$4/255 = 15.62$
1.5	0 to 3	$3/256 = 11.71$
1.28	0 to 2.56	$2.56/256 = 10.00$
1.0	0 to 2	$2/256 = 7.81$
0.5	0 to 1	$1/256 = 3.90$

Table 16: $V_{ref}/2$ relation to V_{in} range.



Note: CS is held low for both the $\overline{RD}$ and $\overline{WR}$ pulses.

Figure 16: Read and write timing for the ADC0804.

Procedure

Activity 1 — Free-running mode

Objective: run the ADC0804 in free-running mode.

1. Connect the ADC0804 as shown in Figure **??** for free-running mode.
2. Vary the input signal from 0 to 5 V using the $10\,\text{k}\Omega$ potentiometer shown in Figure **??** as the ADC's input. The input must not exceed 5 V.
3. Tabulate your results in Table **??**.
4. Compare your hardware result with the value calculated using Equation (1).

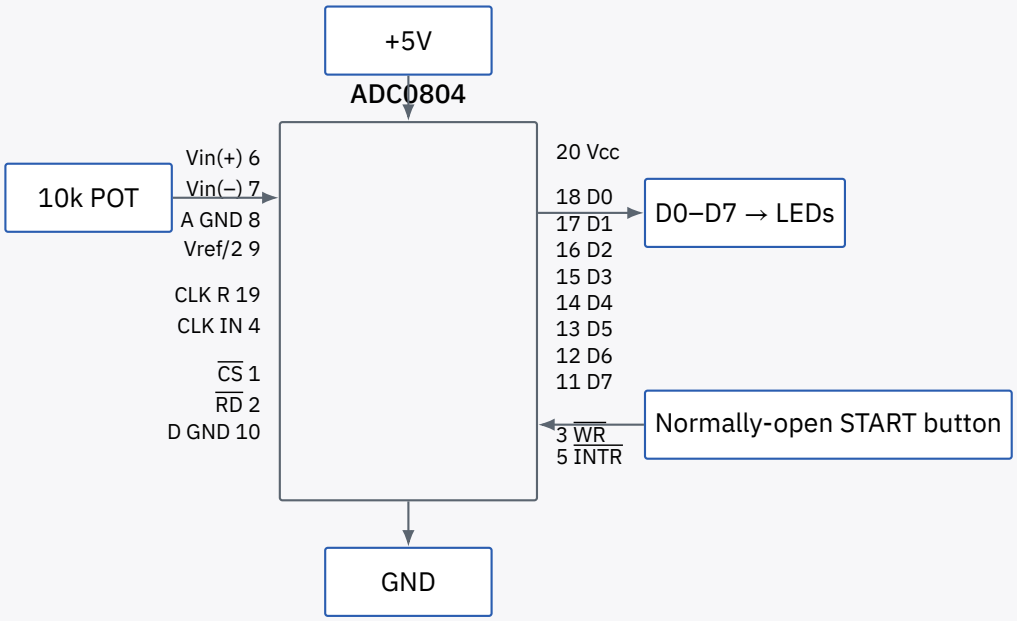


Figure 17: ADC0804 test circuit for free-running mode.

#	Input (V)	Output (hardware)	Output (calculation)
1	0.0		
2	0.5		
3	1.0		
4	1.5		
5	2.0		
6	2.5		
7	3.0		
8	3.5		
9	4.0		
10	4.5		

Table 17: Free-running mode output.

Activity 2 — Interfacing the ADC to the FPGA, displaying on LEDs

Objective: interface the ADC to the FPGA and display the digitised result on LEDs.

Pending

Safety warning (from the original procedure): do not connect the ADC's D0–D7 or $\overline{\text{INTR}}$ pins directly to the DE2 board — doing so will damage the board. These must be buffered through the 74LS245, as described below.

1. Use the circuit from Figure **??**, modified as follows.
2. Connect the $\overline{\text{RD}}$ and $\overline{\text{WR}}$ pins to general-purpose I/O pins on the DE2 board, declared as **outputs**.
3. Connect the $\overline{\text{INTR}}$ and D0–D7 pins to the inputs of a 74LS245. Connect the 74LS245's outputs to general-purpose I/O pins on the DE2 board, declared as **inputs**. Refer to the DE2 user manual for pin locations.
4. In Verilog, generate the $\overline{\text{WR}}$ pulse to start conversion, poll $\overline{\text{INTR}}$ for end of conversion, then pulse $\overline{\text{RD}}$ to read the data on D0–D7.
5. Assign the FPGA's digitised outputs to the LED pins.

6. Compile, simulate, and download the program to the DE2 board.
7. Use the potentiometer to vary the input value.
8. Tabulate your results in Table **??**.
9. Print and submit the Verilog code, simulation result, and experimental results.

#	Input (V)	Output
1	0.8	
2	1.7	
3	2.3	
4	2.8	
5	3.1	
6	3.6	
7	3.9	
8	4.1	
9	4.6	
10	4.8	

Table 18: FPGA-interfaced ADC output.

Activity 3 — Displaying the ADC result on the LCD

Objective: interface the ADC0804 to the DE2 board and display the result on the LCD.

1. Using the same circuit as in Activity 2, write Verilog code to read data from the ADC and display the result on the LCD (see Experiment 5 for the LCD interface).
2. Print and submit the Verilog code and simulation result.

EEM 441 --- Control, Instrumentation & FPGA Laboratory

Experiment 10

Interfacing FPGA to a Digital-to-Analog Converter

Objectives

1. To understand the operation of a digital-to-analog converter (DAC).
2. To interface an FPGA, via the Altera DE2 board, to a DAC.

Equipment / Software Required

- Altera DE2 board
- Multimeter / oscilloscope
- DAC0800 digital-to-analog converter
- Resistors: $10\,\text{k}\Omega$ ($\times 4$), $100\,\Omega$ ($\times 1$)
- Capacitor: $0.01\,\mu\text{F}$

Introduction

A digital-to-analog converter (DAC) converts digital pulses into an analog signal. The first criterion for judging a DAC is its resolution, which depends on the number of binary inputs: the number of distinct analog output levels equals 2^n, where n is the number of data-bit inputs. An 8-bit DAC such as the DAC0800 therefore provides 256 discrete voltage (or current) output levels.
A DAC can be used to generate a sine wave. The output voltage for a given angle θ is:

$$V_{out} = 5\,\text{V} + (5\,\text{V} \times \sin\theta)$$

To find the digital value to send to the DAC for a given angle, multiply the resulting V_{out} by 25.6 (since there are 256 steps across a $10\,\text{V}$ full-scale output).

Procedure

Activity 1 — DAC0800 operation

Objective: understand the operation of the DAC0800.

1. Connect the DAC0800 as shown in Figure **??**.
2. Apply the digital input values at D0–D7 as listed in Table **??**.
3. Measure the resulting analog output and tabulate your results in Table **??**.

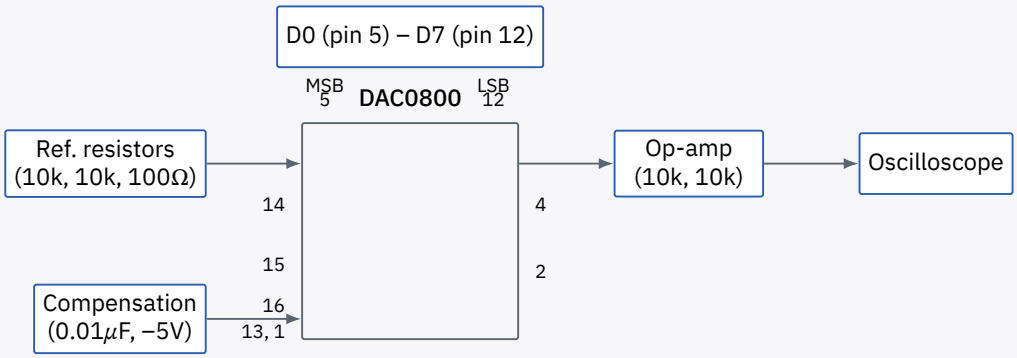


Figure 18: DAC0800 test circuit with op-amp output stage.

Digital value (decimal)	Analog value
0	
10	
60	
80	
100	
128	
180	
200	
220	
255	

Table 19: DAC0800 digital-to-analog conversion results.

Activity 2 — Generating a square wave

Objective: interface the FPGA to the DAC0800 and generate a square wave.

1. Connect the circuit as shown in Figure **??**.
2. Connect D0–D7 of the DAC to general-purpose I/O pins on the DE2 board.
3. Write Verilog code to generate a square wave with a frequency of 100 Hz and a 70% duty cycle.
4. You may use a lookup table, following the same approach as the LCD program in Experiment 5.
5. Print and submit the Verilog code and simulation result.

Activity 3 — Generating a sine wave

Objective: interface the FPGA to the DAC0800 and generate a sine wave.

1. Using the equation in the Introduction, complete Table **??**.
2. Connect the circuit as shown in Figure **??**.
3. Connect D0–D7 of the DAC to general-purpose I/O pins on the DE2 board.
4. Write Verilog code to generate a sine wave with a frequency of 100 Hz.
5. You may use a lookup table, following the same approach as the LCD program in Experiment 5.
6. Print and submit the Verilog code and simulation result.

Angle (°)	$\sin\theta$	V_{out} (V)	Value sent to DAC ($V_{out} \times 25.6$)
0			
30			
60			
90			
120			
150			
180			
210			
240			
270			
300			
330			
360			

Table 20: Angle vs. voltage magnitude for the sine wave.